# Migration as Pathfinding

> *"Deriving the destination is necessary, but designing the journey from the existing system, with legacy, constraints and real teams, is perhaps where architects truly earn their value."* — Loïc Veyssière, written under the article this book grew from

Two teams inherit the same assignment (the audit has spoken, the monolith's checkout component must become its own deployable with its own store) and they sequence it differently. The first team extracts the component now and plans to separate its data later. The second separates the data first, inside the monolith — per-component ownership of tables, one deployable — and extracts along the clean seam afterward. Same starting vector, same target vector. The first team spends eight months operating a shared database astride a network boundary: cross-component writes that were transactions are now a protocol, while the schema still couples both sides: the costs of two positions, the capabilities of neither. The second team never passes through that state at all.

Deltas do not commute. The order of moves is a real decision with real physics, and it is derivable — from the same instruments, pointed at a different question. The derivation picks the *target*; this chapter picks the *route*: a sequence of steps from the vector you hold to the vector the answers force, where every intermediate state is a system you actually run.

## The model

Migration is pathfinding through vector space. **Nodes** are operable vectors: positions per axis, with scopes. **Edges** are deltas: one coherent transformation — an axis move at a scope, a scope split or merge, or a scaffolding step. **Edge weights** are six cost-and-risk indicators, deliberately qualitative: the reversibility of the step; the feasibility of the intermediate state while the system runs through the change; the coupling cost of coordinated changes elsewhere; the duration spent in dual state; the dependency on prior transformations; and the failure-mode amplification an irreversible step carries — which is what earns a step its mitigation plan.

Qualitative weights change how paths are compared: argument, never arithmetic. A path is rejected by *naming the indicator that kills it* — prune-mode reasoning applied to routes — and never by a weighted sum, because a weighted sum over incommensurable risks is taste with decimal places.

Three **path constraints** govern every route, and they are where the opening story lives. Each intermediate state must be **operable**: staffed, debuggable, on-call-able; the intermediate *is* production, usually for months. Each must be **affordable**: dual-running pays the boundary cost from Chapter 3 in duplicate, genuinely if temporarily. And each must be **calendar-feasible**: businesses have immovable walls — filing seasons, year-end closes, retail peaks — and a delta whose dual-state period would span a wall is infeasible even when the delta itself is small, because the frozen weeks must never contain a half-migrated system. *When* is a path property, and the payroll valida-

tion supplied the canonical instance: a one-axis merge, trivial as a delta, that simply cannot transit filing season.

## Non-commutativity, worked

The opening pair is the persistence-and-topology case: extract-then-separate transits the infeasible node (the shared store across a network boundary — the distributed monolith, named at last and defined precisely: a topology move whose consistency decayed to protocol while its schema coupling stayed); separate-then-extract keeps every intermediate operable. Same endpoints; the order was the entire decision.

A second shape recurs so often it deserves its own name: **the priced scaffold**. A separated read model needs a change feed. If an event substrate is on the path anyway, sequence it first and the projections ride the published facts: the second delta shrinks. Projections first means building change-capture scaffolding the later substrate move will demolish. That order is *allowed* — sometimes the read path's deadline genuinely comes first — but the scaffold must enter the plan priced as a whole edge: built, operated, and torn down. Pathfinding's contribution is making the throwaway explicit at planning time instead of discovering it as "wasted work" in a retrospective. Chapter 8 called this the scaffolding step from the failure side; here it is a legitimate purchase with a price tag.

And the third shape is triage: deciding what kind of move a "migration" even is, using the vector lens Chapter 7 validated on Discord's two store migrations. A **product swap** moves no axis: same position, different vendor, procurement with operational care, and the documented easy case. A **single-axis delta** at one scope is the ordinary edge. A **compound delta** — several axes at once — is where programs go to die, and the pathfinding response is decomposition: find an order of single-axis deltas whose intermediates are all feasible. When no such order exists, the route needs a scaffolding node, or the target needs Chapter 8's renegotiation menu, because "we cannot get there operably from here" is a contradiction between the target and the calendar, and it prices like any other.



## Ordering principles

Dependencies impose a partial order — some deltas are prerequisites for others. Inside that order, one principle does most of the sequencing work, and it is the principle Part I never needed: **irreversibility orders commitment.** Reversible deltas go early — they are experiments the path can retreat from; the hard-to-reverse delta goes as late as the dependencies allow, because an early irreversible step forecloses every alternative route while information is still arriving. Greenfield derivations never fire this rule; every transformation fires it constantly, which is why it lives in this chapter.

Field experience corroborates the rule from an independent direction. Poltorak's catalog (next section) observes, three separate times, in nearly identical words, that certain infrastructure layers, once adopted, are "unlikely to be removed": middleware, shared repositories, proxies are practical one-way doors. The catalog states it as observation; the indicator set explains it — those layers' removal costs amplify with ev-

erything built atop them — and the sequencing consequence is the same either way: treat one-way doors as late-path commitments.

## The edge list, and what catalogs price

A pathfinding model wants an edge list: which transitions between positions are known, with what prerequisites and costs. This book's edge list is adapted, with attribution and thanks, from Denys Poltorak's *Architectural Metapatterns* (CC BY 4.0) — whose "Evolutions of Architectures" appendix catalogs seventeen-odd transitions between named shapes, each with prerequisites, benefits, and drawbacks, several with explicit staged decompositions. Revoiced into six-axis vocabulary, his transitions map cleanly onto edges: his prerequisites read as destination-fitness conditions, his drawbacks as always-on costs and failure-mode amplifications.

The mapping also exposed a boundary worth stating precisely, because it defines this chapter's contribution relative to the catalogs. **Transition catalogs price destinations; a migration must price the crossing.** A prerequisite like "the data can be split into semi-independent parts" gates whether the target fits the domain. It says nothing about whether the system stays operable *while the split is in progress*. Intermediate-state feasibility, dual-state duration, calendar walls — the properties that killed the first team's route in the opening — have almost no counterpart in any catalog's fields, and could not: they are properties of *routes*, and catalogs enumerate *places*. Both prices are real. Pay the destination price from the catalogs; derive the crossing price here.

## The loop, closed

Each executed delta leaves its decision record — position, forcing answers, costs accepted, revisit trigger — so the path documents itself as it is walked; the inheritor receives the route, and every route survives its droughts better than its destination survives alone. Upstream, the rhythm from Chapter 9 keeps running: the audit detects *that* a step is due, this chapter walks it, and answers changing mid-path re-enter the derivation like answers changing anywhere else: a path is as recomputable as the target it aims at, and long programs that cannot absorb a changed answer mid-route are waterfalls wearing migration branding.

Everything in Part III so far assumed a defensible starting point: a system whose current vector is *known*, whose answers are on file, whose history is legible. The next chapter drops that assumption the way reality does — a system built by people who are gone, for reasons nobody recorded, audited by an institution with no stake in flattering anyone — and runs the entire method against the hardest starting position there is.

*Rules exercised: deltas and the six indicators · reject-by-named-indicator · the three path constraints (operable, affordable, calendar-feasible) · non-commutativity · the priced scaffold · migration triage by the vector lens · irreversibility-orders-commitment · destination price vs crossing price.*